

KGQA Hierarchical Project: Exhaustive Line-by-Line Technical Report

Every file. Every line. Every design decision. No exceptions.

TABLE OF CONTENTS

PART 1: THE PROBLEM

- 1.1 What is KGQA?
- 1.2 Why Relations, Not Entities?
- 1.3 The Dataset: ComplexWebQuestions (CWQ)
- 1.4 The Complete Experiment Progression & Real Accuracy Numbers

PART 2: SHARED INFRASTRUCTURE (Read These First)

- 2.1 `shared/encoder.py` — The Question-to-Vector Brain
- 2.2 `utils/sparql_parser.py` — The SPARQL Truth Extractor
- 2.3 `shared/kg_loader.py` — The Physical Knowledge Graph
- 2.4 `shared/metrics.py` — Hits@K and MRR
- 2.5 `utils/metrics.py` — Accuracy helpers
- 2.6 `shared/cwq_parser.py` — CWQ Data Loader Wrapper
- 2.7 `models/model.py` — The First Entity-Aware Architecture Prototype

PART 3: DATA PIPELINE FILES

- 3.1 `data/build_supervision.py` — CWQ Hop-level Supervision (v1)
- 3.2 `data/build_supervision_with_entities.py` — CWQ Supervision + Entity IDs
- 3.3 `data/build_universal_vocab.py` — The Universal Vocab Builder (Exp 10)

PART 4: TRAINING — THE EXPERIMENT CHAIN

- 4.1 `train/train_relation_flat.py` — Experiment 0: The Flat Baseline
- 4.2 `train/exp1_domain_baseline.py` — Experiment 1: Domain-Aware BERT
- 4.3 `train/exp2_cpd.py` — Experiment 2: Contrastive Path Decoding v1
- 4.4 `train/exp3_pct.py` — Experiment 3: Path Confidence Tracker
- 4.5 `train/exp4_chcp.py` — Experiment 4: Cross-Hop Coherence Planning
- 4.6 `train/exp5_rlmc.py` — Experiment 5: RL Meta-Constraint (Prototype/Blueprint)
- 4.7 `train/exp6_unified.py` — Experiment 6: Unified Adaptive Planner
- 4.8 `train/exp7_roberta.py` — Experiment 7: Scaling to RoBERTa-Large
- 4.9 `train/exp8_cpd_roberta.py` — Experiment 8: Contrastive RoBERTa
- 4.10 `train/exp9_rlmc.py` — Experiment 9: RL Meta-Constraint Agent (SOTA)
- 4.11 `train/exp10_universal.py` — Experiment 10: Universal Multi-Dataset Agent

PART 5: EVALUATION FILES

- 5.1 `eval/e2e_evaluate.py` — The End-to-End Evaluator (All Models)
- 5.2 `eval/universal_eval.py` — The Universal Dataset Evaluator
- 5.3 `utils/verify.py` — Quick Checkpoint Verifier

PART 6: RESULTS & PRODUCTION

- 6.1 `results.md` — Actual Measured Accuracy Numbers
- 6.2 Production Entity Linking Strategy
- 6.3 Mathematical Glossary

PART 1: THE PROBLEM

1.1 What is KGQA?

Knowledge Graph Question Answering (KGQA) is the task of answering a natural language question by navigating a structured Knowledge Graph (KG).

A Knowledge Graph is a database of facts stored as "triples": (Subject Entity) --[Relation]--> (Object Entity)

Real examples from Freebase (the KG used in this project): "Barack Obama" --[nationality]--> "United States"
"The Dark Knight" --[directed_by]--> "Christopher Nolan" "USA" --[president]--> "Joe Biden"

A question like "Who directed the film starring the actor born in Chicago?" requires a chain of such facts to be traversed in the correct order.

1.2 Why Relations, Not Entities?

This is CRITICAL. The model NEVER directly predicts "what is the answer entity." Here is why:

SCALE PROBLEM: Freebase has ~100,000,000 (100 Million) entities. A neural network cannot output "which of 100M things is the answer." Even if trained, the 100M-class classifier would have 100M output neurons — that is billions of parameters just for the output layer.

SOLUTION: Freebase has only ~900-1000 UNIQUE RELATION TYPES. That is a manageable classification problem.

STRATEGY: Step 1: We are GIVEN the "Start Entity" (called the "topic entity") from the dataset JSON (e.g., `item["topic_entity"] = "m.09c7w0"`). Step 2: Our model PREDICTS the SEQUENCE OF RELATIONS to traverse. Example output: `["government.country.president", "people.person.children"]` Step 3: We EXECUTE this sequence on the physical KG starting from the start entity. Step 4: The entities we arrive at ARE THE ANSWER.

EXAMPLE: Question: "Who are the children of the current US president?" Topic Entity: `m.09c7w0` (United States of America)

```
Model predicts: ["government.country.president", "people.person.children"]
```

```
Execution:
```

```
Set_0 = {m.09c7w0} (USA)
```

```
Set_1 = KG.traverse(Set_0, "government.country.president")
        = {m.02mjmr, m.0bymv, ...}    (Joe Biden, Barack Obama, ...)
Set_2 = KG.traverse(Set_1, "people.person.children")
        = {m.04t4sx, m.0bymv2, ...}    (Their kids)
```

Output: Set_2 = The answer entities

1.3 The Dataset: ComplexWebQuestions (CWQ)

CWQ contains ~35,000 training / 3,500 dev / 3,500 test questions. Questions require 1 to 4 hops through Freebase to answer. Each item in the JSON contains: - "question": The English question text - "sparql": A SPARQL query that, when executed on Freebase, yields the answer - "answers": A list of gold answer entities

The SPARQL is our "truth." We parse it to extract the gold relation path.

1.4 Complete Experiment Progression & Real Accuracy Numbers

Data from results.md (actual measured results):

Experiment	Model	Dev Hits@1	Test Hits@1	Key Innovation
Exp 0	Flat BERT MLP	32.53%	31.83%	Baseline — blind classification
Exp 3	PCT Multi-head	30.85%	26.85%	Confidence heads — slightly hurt raw accuracy
Exp 4	CHCP Transformer	56.20%	55.50%	Hierarchical hop planning (+25% jump!)
Exp 4-RL	CHCP + RL (failed)	23.50%	24.05%	RL on wrong backbone — hurt performance
Exp 6	Unified	53.60%	51.70%	Combines Exp 3 + 4
Exp 7	RoBERTa-Large	56.40%	57.28%	Bigger model, better encoder
Exp 8	CPD + RoBERTa	58.20%	56.76%	Hard-negative contrastive tuning
Exp 9	RLMC (execution)	—	76.66%	RL on the right backbone, graph execution

Key observation: Exp 0→4 showed a +23% jump. Exp 7→9 showed a further +20% jump. The two biggest gains came from: (1) hierarchical planning and (2) RL-based execution.

PART 2: SHARED INFRASTRUCTURE

2.1 FILE: shared/encoder.py

```

import torch, torch.nn as nn
from transformers import BertModel, BertTokenizer

class QuestionEncoder(nn.Module):
    def __init__(self, model_name="bert-base-uncased"):
        super().__init__()
        self.bert = BertModel.from_pretrained(model_name)

    def forward(self, input_ids, attention_mask):
        outputs = self.bert(input_ids=input_ids, attention_mask=attention_mask)
        return outputs.pooler_output

    @property
    def output_dim(self):
        return self.bert.config.hidden_size

```

LINE BY LINE:

`BertModel.from_pretrained("bert-base-uncased")`: Downloads (or loads from cache) a pre-trained BERT model. BERT = Bidirectional Encoder Representations from Transformers (Google, 2018). "bert-base-uncased" = 12 transformer layers, 768 hidden units, 110M parameters. "Uncased" = lowercased input ("Obama" and "obama" are treated identically). Pre-trained on: Wikipedia (2.5B words) + BooksCorpus (800M words). BERT already "knows" English grammar, common facts, entities, and relationships.

`input_ids`: The question converted from text to a tensor of integer token IDs. Example: "Where was Obama born?" Tokens: ["[CLS]", "where", "was", "obama", "born", "?", "[SEP]"] IDs: [101, 2073, 2001, 29853, 2141, 1029, 102] Shape: [batch_size, max_sequence_length]

`attention_mask`: A binary tensor of same shape as `input_ids`. 1 for real tokens, 0 for [PAD] tokens added to make all sequences the same length. BERT ignores tokens where `attention_mask = 0`. Example (if padded to length 10): [1, 1, 1, 1, 1, 1, 1, 0, 0, 0]

`outputs.pooler_output`: BERT outputs a hidden state for EVERY token. Shape: [batch, seq_len, 768]. The `pooler_output` is specifically the [CLS] token's representation, passed through a linear layer + tanh. Shape: [batch, 768]. [CLS] is designed to capture the WHOLE SENTENCE meaning, not just one word. This single 768-dim vector is "the meaning of the question" in mathematical form.

`output_dim` property: Returns 768 for bert-base, 1024 for roberta-large. The downstream layers (MLPs, Transformers) use this to know their input size.

```

class PathEncoder(nn.Module):
    def __init__(self, relation_dim, hidden_dim):
        super().__init__()
        self.lstm = nn.LSTM(relation_dim, hidden_dim, batch_first=True,
                             bidirectional=True)

    def forward(self, relation_embeddings):
        output, (hn, cn) = self.lstm(relation_embeddings)

```

```
path_repr = torch.cat((hn[-2,:,:], hn[-1,:,:]), dim=1)
return path_repr
```

`nn.LSTM(relation_dim, hidden_dim, batch_first=True, bidirectional=True)`: An LSTM (Long Short-Term Memory) network. - `relation_dim`: input size per step (size of one relation embedding) - `hidden_dim`: size of the hidden state at each step - `batch_first=True`: input shape is [batch, seq, features] (not [seq, batch, features]) - `bidirectional=True`: runs TWO LSTMs — one left-to-right, one right-to-left

WHY BIDIRECTIONAL?

If path = ["film.film.actor", "people.person.nationality"]:

Forward LSTM reads hop1 → hop2. It knows: "after actor, we look at nationality."

Backward LSTM reads hop2 → hop1. It knows: "nationality comes from an actor relation."

The combined representation captures BOTH the flow AND the end-goal of the path.

`hn[-2,:,:]` and `hn[-1,:,:]`: `hn` is the final hidden state tensor. Shape: [num_layers * num_directions, batch, hidden_dim]. For bidirectional with 1 layer: shape = [2, batch, hidden_dim]. `hn[-2]` = final state of the FORWARD direction. `hn[-1]` = final state of the BACKWARD direction. Concatenating them: [batch, hidden_dim*2] = the full path summary.

2.2 FILE: utils/sparql_parser.py

This file is the "Gold Label Extractor." It takes a raw SPARQL string from CWQ and extracts the exact sequence of Freebase relations that leads to the answer.

```
TRIPLE_PATTERN = re.compile(
    r'(\?w+|ns:[mg]\.[\\w\\d_]+)\s+ns:(\\w\\d\\. _)+\s+(\?w+|ns:[mg]\.[\\w\\d_]+)\s*\. '
)
```

regex breakdown character by character: `(\?w+|ns:[mg]\.[\\w\\d_]+)`: Match either: Option A: `\?w+` = a SPARQL variable like `?x`, `?y`, `?person` Option B: `ns:[mg]\.[\\w\\d_]+` = a Freebase entity like `ns:m.09c7w0` or `ns:g.abc123`

```
`\s+ns:(\\w\\d\\. _)+`:
```

One or more whitespace chars, then the relation prefixed with "ns:"

The capture group ``(\\w\\d\\. _)+`` captures: "government.country.president"

```
`\s+` (same pattern for object) and `\s*\.` for the trailing dot.
```

```
def extract_triples(sparql):
    matches = TRIPLE_PATTERN.findall(sparql)
    for subj, rel, obj in matches:
```

```
triples.append((subj, rel, obj))
return triples
```

Input SPARQL: SELECT DISTINCT ?x WHERE { ns:m.09c7w0 ns:government.country.president ?y . ?y ns:people.person.children ?x . }

findall extracts: [("ns:m.09c7w0", "government.country.president", "?y"), ("?y", "people.person.children", "?x")]

```
def build_graph(triples):
    graph = defaultdict(list)
    for subj, rel, obj in triples:
        graph[subj].append((obj, rel, +1)) # Forward edge
        graph[obj].append((subj, rel, -1)) # Backward edge
    return graph
```

Builds a bidirectional in-memory adjacency list from the SPARQL triples. This is a LOCAL graph of the question's structure — NOT the full Freebase KG. Direction +1 = forward (subj → obj). Direction -1 = backward (obj ← subj). Backward edges are critical because some answers require traversal against the arrow.

```
def find_entity_constants(triples):
    for subj, rel, obj in triples:
        if subj.startswith("ns:m.") or subj.startswith("ns:g."):
            entities.add(subj)
```

Freebase entity IDs start with "ns:m." (machine ID) or "ns:g." (Google ID). SPARQL variables start with "?". This function finds the KNOWN starting entities.

```
def find_reasoning_path(sparql):
    triples = extract_triples(sparql)
    graph = build_graph(triples)
    answer_var = find_answer_variable(sparql) # Usually "?x"
    entity_constants = find_entity_constants(triples)

    for start in entity_constants:
        queue = deque([(start, [])])
        while queue:
            node, path = queue.popleft()
            if node == answer_var:
                return path # FOUND!
            visited.add(node)
            for neighbor, rel, direction in graph[node]:
                new_path = path + [(node, rel, direction, neighbor)]
                queue.append((neighbor, new_path))
    return None
```

BFS (Breadth-First Search) execution trace for the example SPARQL:

```

Start: entity_constants = {"ns:m.09c7w0"}

Iteration 1:
  node = "ns:m.09c7w0", path = []
  Is it "?x"? NO.
  Neighbors: [("y", "government.country.president", +1)]
  Push: ("y", [("ns:m.09c7w0", "government.country.president", +1, "y")])

Iteration 2:
  node = "y", path = [step1]
  Is it "?x"? NO.
  Neighbors: [("ns:m.09c7w0", "government.country.president", -1),
              ("x", "people.person.children", +1)]
  Push: ("x", [step1, ("y", "people.person.children", +1, "x")])

Iteration 3:
  node = "x", path = [step1, step2]
  Is it "?x"? YES! Return path.

```

Result: [("ns:m.09c7w0", "government.country.president", +1, "y"), ("y", "people.person.children", +1, "x")]

From this, we extract: relations = ["government.country.president", "people.person.children"] These become the GOLD LABELS for training.

2.3 FILE: shared/kg_loader.py

This is the physical, in-memory representation of the Knowledge Graph. During execution (and graph hard masking), every relation prediction is checked against this structure.

```

class KnowledgeGraph:
    def __init__(self):
        self.forward = defaultdict(list)    # subject → [(relation, object)]
        self.backward = defaultdict(list)   # object → [(relation, subject)]
        self.entities = set()
        self.relations = set()

```

Two dictionaries: forward and backward. forward["Obama"] = [("nationality", "USA"), ("profession", "politics"), ...] backward["USA"] = [("nationality", "Obama"), ("nationality", "Elon Musk"), ...]

WHY BACKWARD? Some SPARQL paths traverse AGAINST the edge direction: "Who published a book about nuclear physics?" Path: "nuclear_physics" <--[subject_of]-- "Book" --[published_by]--> "Publisher" At hop 2, we're at "Book" and need to go BACKWARD along "subject_of". Without backward edges, we'd miss half the graph.

```

def add_triple(self, subj, rel, obj):
    self.forward[subj].append((rel, obj))
    self.backward[obj].append((rel, subj))

```

```

self.entities.add(subj)
self.entities.add(obj)
self.relations.add(rel)

```

Every time a triple is added, BOTH forward and backward are updated.

```

def get_neighbors(self, entity):
    neighbors = []
    for rel, tgt in self.forward.get(entity, []):
        neighbors.append((rel, +1, tgt)) # Forward edges
    for rel, tgt in self.backward.get(entity, []):
        neighbors.append((rel, -1, tgt)) # Backward edges
    return neighbors

```

THIS IS THE GRAPH HARD MASKING FUNCTION. When we are at entity E and want to know which relations are PHYSICALLY POSSIBLE: possible_rels = {rel for rel, direction, tgt in kg.get_neighbors(E)}

Any relation NOT in possible_rels gets its logit set to -infinity during decoding. This prevents the model from predicting "chemistry.element.symbol" for a person entity. The model can ONLY pick relations that actually EXIST at the current entity.

NOTE: The masking is about RELATION EXISTENCE, not entity existence. The entity (E) is always known and exists. The question is what RELATIONS it has.

```

def traverse(self, start_entity, relation_path):
    current = {start_entity} # START: set of one entity
    for rel, direction in relation_path:
        next_entities = set()
        for entity in current: # For EVERY entity in the current wave
            if direction == +1:
                for r, tgt in self.forward.get(entity, []):
                    if r == rel:
                        next_entities.add(tgt) # ADD to next wave
            else: # Backward direction
                for r, tgt in self.backward.get(entity, []):
                    if r == rel:
                        next_entities.add(tgt)
        current = next_entities
        if not current:
            break # Dead end – stop immediately
    return current # Final wave = ANSWER ENTITIES

```

SET-BASED REASONING — The Key to Handling Branching:

The model never "picks one entity" at each step. It maintains a wave. Example with branching:

Question: "All scientists that a US president has met?"

Topic Entity: USA (m.09c7w0)

Path: ["government.country.president", "people.person.met"]

Step 0: current = {USA} (1 entity)

Step 1 (country.president):

USA has: Obama, Biden, Trump, Clinton, Bush, Carter...

current = {Obama, Biden, Trump, Clinton, Bush, Carter} (43 entities)

Step 2 (person.met):

Obama met: Hawking, Merkel, Xi, Sagan's estate...

Biden met: ...

ALL 43 presidents' meetings are unioned:

current = {Hawking, Merkel, Sagan, Xi, ...} (potentially hundreds)

The GOLD ANSWER is then determined by further filtering (e.g., if there's a third hop "people.person.profession = scientist").

If current becomes empty at any step: the path is a DEAD END. The entire traversal returns empty set = wrong answer.

2.4 FILE: shared/metrics.py

```
def hits_at_k(preds, targets, k=1):
    topk = torch.topk(preds, k=k, dim=-1).indices # Get indices of top-k logits
    correct = (topk == targets.unsqueeze(1)).any(dim=1).float().sum()
    return correct.item()
```

`torch.topk(preds, k=1)`: For each sample in the batch, find the k highest logit values and return their indices. Example: `preds = [[2.1, 8.3, 1.0, 5.5]]`, `k=3` Returns indices: `[[1, 3, 0]]` (relations 1, 3, 0 are the top-3)

`.any(dim=1)`: For each sample, check if ANY of the top-k predictions matches the target. This turns the [batch, k] comparison into a [batch] boolean tensor.

Hits@1: Is the TOP prediction correct? (Most strict) **Hits@3**: Is the correct relation anywhere in the TOP 3 predictions? (More lenient)

```
def mean_reciprocal_rank(preds, targets):
    sorted_indices = torch.argsort(preds, dim=-1, descending=True)
    ranks = (sorted_indices == targets.unsqueeze(1)).nonzero(as_tuple=True)[1] + 1
    mrr = (1.0 / ranks.float()).sum()
    return mrr.item()
```

MRR (Mean Reciprocal Rank): For each sample, find the RANK of the correct answer (1=top, 2=second, etc.)
 Reciprocal rank = 1/rank. MRR = average of reciprocal ranks across all samples. If correct is always ranked 1st: MRR = 1.0 (perfect). If correct is always ranked 10th: MRR = 0.1 (poor).

2.5 FILE: utils/metrics.py

```
def accuracy(logits, targets):
    preds = torch.argmax(logits, dim=-1)          # Pick the single highest-scoring
    class
    correct = (preds == targets).float().sum()
    return correct / len(targets)                  # Fraction correct

def topk_accuracy(logits, targets, k=3):
    topk = torch.topk(logits, k=k, dim=-1).indices
    correct = (topk == targets.unsqueeze(1)).any(dim=1).float().sum()
    return correct / len(targets)
```

accuracy: Standard Top-1 accuracy. Fraction of samples where `argmax` = true label. **topk_accuracy**: Fraction of samples where true label is within top-k predictions. These are simpler utility functions compared to `shared/metrics.py`. Note: Unlike `shared/metrics.py`, these return a FRACTION not a COUNT.

2.6 FILE: shared/cwq_parser.py

```
class CWQParser:
    def __init__(self, file_path):
        self.file_path = file_path
        self.data = []

    def load(self):
        with open(self.file_path, "r", encoding="utf-8") as f:
            self.data = json.load(f)
        return self.data

    def get_parsed_samples(self):
        samples = []
        for item in self.data:
            hops = extract_hop_supervision(item["question"], item["sparql"])
            triples = extract_triples(item["sparql"])
            samples.append({
                "id": item.get("ID", ""),
                "question": item["question"],
                "sparql": item["sparql"],
                "hops": hops,
                "triples": triples
            })
        return samples
```

A simple wrapper class around the JSON data loading and SPARQL parsing. `load()`: reads the CWQ JSON file into memory. `get_parsed_samples()`: for every item, runs SPARQL parsing and returns structured data. - **hops**: the list of (question, relation, direction) supervision tuples - **triples**: the raw extracted (subj, rel, obj) triples for KG construction This class is used for quick data inspection and preprocessing scripts.

2.7 FILE: models/model.py

```
class DomainOnlyModel(nn.Module):
    def __init__(self, question_dim, num_entities, entity_dim, hidden_dim,
num_domains):
        self.entity_embedding = nn.Embedding(num_entities, entity_dim)
        self.fc = nn.Sequential(
            nn.Linear(question_dim + entity_dim, hidden_dim),
            nn.GELU(),
            nn.LayerNorm(hidden_dim)
        )
        self.domain_head = nn.Linear(hidden_dim, num_domains)

    def forward(self, question_emb, entity_ids):
        e_emb = self.entity_embedding(entity_ids)          # Look up entity embedding
        h = torch.cat([question_emb, e_emb], dim=-1)      # Concatenate question +
entity
        h = self.fc(h)
        logits = self.domain_head(h)
        return logits
```

This is an EARLY PROTOTYPE — not used in final experiments. Key innovation: it explicitly uses the ENTITY as part of the model input.

`nn.Embedding(num_entities, entity_dim)`: A lookup table mapping entity IDs to learnable vectors. If Obama has ID=5, `entity_embedding(5)` returns a 128-dim learned vector. These vectors are trained to capture "what kind of entity am I?" Obama (politician) will cluster close to other politicians, far from locations.

`torch.cat([question_emb, e_emb], dim=-1)`: CONCATENATION of question meaning + entity meaning into one long vector. If question_dim=768 and entity_dim=128: output is 896-dim. This gives the downstream MLP both "what is being asked?" AND "where are we starting from?"

WHY WASN'T THIS USED IN THE FINAL EXPERIMENTS? The experiments moved towards injecting entity TEXT (not IDs) into the question. Instead of separate entity embeddings, Exp 10 uses entity name prepended to question. This is more generalizable to new, unseen entities (no embedding = no OOV problem).

PART 3: DATA PIPELINE FILES

3.1 FILE: data/build_supervision.py

PURPOSE: Creates per-HOP training samples from CWQ. Each sample is one hop of one question. For a 2-hop question, this produces 2 samples (one per hop).

```
def extract_split_hops(cwq_data):
    for item in cwq_data:
        hops = extract_hop_supervision(item["question"], item["sparql"])
        for hop in hops:
```

```

        rel = hop["relation"]
        dom = get_domain(rel)                    # "people.person.nationality" →
"people"
        hop_samples.append({
            "question": question,                # SAME question for EVERY hop
            "relation": rel,                     # The SPECIFIC relation at THIS
hop
            "domain": dom                        # The domain of this hop's
relation
        })

```

IMPORTANT: Every hop of a multi-hop question uses the SAME original question text. This means the model must learn to use different parts of the question for different hops. For "Who are the children of the US president?": Hop 1 sample: question="Who are the children...", target=government.country.president Hop 2 sample: question="Who are the children...", target=people.person.children

```

def build_vocab(global_relation_set, global_domain_set):
    relation2id = {r: i for i, r in enumerate(sorted(global_relation_set))}
    domain2id = {d: i for i, d in enumerate(sorted(global_domain_set))}
    relation_to_domain = torch.zeros(len(relation2id), dtype=torch.long)
    for r, r_id in relation2id.items():
        relation_to_domain[r_id] = domain2id[get_domain(r)]
    return relation2id, domain2id, relation_to_domain

```

`sorted(global_relation_set)`: Alphabetically sorts the set of all relation strings. Then enumerates them to create a deterministic mapping. "biology.organism.habitat" → 0, "education.degree.level" → 1, ... This ensures REPRODUCIBLE and CONSISTENT IDs across runs.

`relation_to_domain`: A tensor of shape [num_relations] where each entry is the domain ID.
`relation_to_domain[312] = 14` means "relation 312 belongs to domain 14 (people)" Used during masked relation prediction: knowing domain → filter irrelevant relations.

```

torch.save(relation2id, "data/processed/relation2id.pt")

```

`torch.save` on a Python dictionary: saves it as a PyTorch binary file. Can be loaded later with `torch.load("relation2id.pt")` → returns the dictionary.

3.2 FILE: data/build_supervision_with_entities.py

PURPOSE: Same as 3.1, but ALSO extracts entity IDs at each hop. Creates richer supervision: (question, entity, relation, domain) per hop.

```

for node, rel, direction, next_node in path:
    entity_set.add(node)
    hop_samples.append({
        "question": question,

```

```

    "entity": node,          # NEW: The entity at THIS hop position
    "relation": rel,
    "domain": get_domain(rel)
  })

```

node is the entity ID at step *h* of the reasoning path. For the "US president" example: Hop 1: entity = "ns:m.09c7w0" (USA), relation = "government.country.president" Hop 2: entity = "?y" (a variable — the intermediate entity), relation = "people.person.children"

The entity information was intended for the DomainOnlyModel prototype. Later experiments moved away from entity IDs in training, using entity NAME injection instead.

3.3 FILE: data/build_universal_vocab.py

PURPOSE: Processes ALL THREE datasets (CWQ, MetaQA, WebQSP) and builds a UNIFIED vocabulary. This is the data pipeline for Experiment 10 (currently running).

```

def _bfs_relations(kg, start, targets, max_hops):
    target_set = set(targets)
    if start in target_set: return []
    queue = deque([(start, [])])
    visited = {start}
    while queue:
        node, path = queue.popleft()
        if len(path) >= max_hops: continue
        for rel, neighbor in kg.get(node, []):
            new_path = path + [rel]
            if neighbor in target_set:
                return new_path          # Return path of relation NAMES
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append((neighbor, new_path))
    return None

```

This BFS is specifically for MetaQA and WebQSP. Unlike CWQ (which has SPARQL), MetaQA has a raw KB file and question text. We must DISCOVER the relation path by BFS from the topic entity to the answer.

max_hops controls depth. For 1-hop MetaQA: max_hops=1. For 3-hop: max_hops=3.

```

def parse_cwq(self):
    for split in ['train', 'dev', 'test']:
        for item in data:
            path_hops = find_reasoning_path(item['sparql'])
            rels = [hop[1] for hop in path_hops]          # Extract relation
            strings

            topic = path_hops[0][0].replace('ns:', '')    # Clean entity ID
            gold_answers = {'m.' + a['answer_id']...}     # Clean answer IDs
            samples.append({
                'question': item['question'],

```

```

        'topic_entity': topic,
        'relations': rels,
        'gold_answers': list(gold_answers),
        'dataset': 'cwq',
        'num_hops': len(rels)
    })
    json.dump(samples, file)
    immediately

```

Dataset tag

Stream to disk

MEMORY EFFICIENCY: We write processed data to disk IMMEDIATELY after processing each split. This avoids holding the entire 43MB cwq_train.json AND processed results in memory simultaneously.

```

def parse_metaqa(self):
    kb_path = 'data/metaqa/kb.txt'
    kg = defaultdict(list)
    for line in f:
        s, r, o = line.strip().split('|')
        kg[s].append((r, o))
        kg[o].append((r + '_inv', s))

```

INVERSE EDGES with "_inv" suffix

MetaQA's KB is a text file with 3-column tab-separated triples. The key innovation here: for EVERY forward edge (s, r, o), we also add a backward edge (o, r_inv, s) where r_inv = original relation name + "_inv" suffix. This lets BFS traverse the graph in BOTH directions without a special direction flag.

```

entity_pat = re.compile(r'\[(.+?)\]')
match = entity_pat.search(q_raw)
topic = match.group(1)

```

The [bracketed entity] in the question

MetaQA questions embed the topic entity in square brackets: "What movies did [Tom Hanks] star in?" The regex extracts "Tom Hanks" as the topic entity.

```

def parse_webqsp(self):
    for q_ent in q_entities:
        res = _bfs_relations(local_kg, q_ent, a_entities, max_hops=2)
        if res is not None:
            gold_rels = res
            start_ent = q_ent
            break

```

WebQSP provides a "graph" field: pre-extracted graph triples for each question. We build a LOCAL KG from these triples, then BFS from question entity to answer entity. This gives us REAL FREEBASE RELATION NAMES instead of synthetic ones. Max hops = 2 because WebQSP is a simpler, 1-2 hop dataset.

PART 4: TRAINING — THE EXPERIMENT CHAIN

4.1 FILE: train/train_relation_flat.py — EXPERIMENT 0

THE ORIGINAL PROTOTYPE. The simplest possible approach.

```
class RelationClassifier(nn.Module):
    def __init__(self, input_dim, hidden_dim, num_relations):
        self.net = nn.Sequential(
            nn.Linear(input_dim, hidden_dim),
            nn.GELU(),
            nn.LayerNorm(hidden_dim),
            nn.Linear(hidden_dim, num_relations)
        )
    def forward(self, x):
        return self.net(x)
```

NOTE: This experiment did NOT use BERT at training time. The input `x` was PRE-COMPUTED BERT embeddings loaded from disk as tensors. This was done for speed: BERT inference is slow. By pre-computing and saving embeddings, training the MLP itself was much faster.

`nn.Linear(input_dim=768, hidden_dim=1024)`: Matrix multiplication: $\text{output} = \text{input} \times W^T + b$. W has shape $[1024, 768]$. b has shape $[1024]$. This learns to "project" the 768-dim BERT space into a "thinking space" of 1024 dims.

`nn.GELU()` (Gaussian Error Linear Unit): $\text{output} = x * 0.5 * (1 + \tanh(\sqrt{2/\pi} * (x + 0.044715 * x^3)))$. A smooth, differentiable version of ReLU. Does NOT hard-clip at 0. Smoothness matters for gradient flow: the gradient is never zero for any input.

`nn.LayerNorm(hidden_dim)`: For EACH sample independently: $\text{mean} = \text{average of all 1024 values in that sample's vector}$, $\text{std} = \text{standard deviation of those values}$, $\text{output} = (\text{input} - \text{mean}) / \text{std} * \gamma + \beta$. Where γ (gamma) and β (beta) are LEARNABLE parameters initialized to 1 and 0. WHY? Without normalization, the magnitude of layer outputs grows or shrinks across training steps, causing "covariate shift." LayerNorm prevents this.

`nn.Linear(hidden_dim=1024, num_relations=916)`: The final classification layer. Outputs ONE logit per relation in the vocabulary. These 916 raw numbers are the LOGITS.

```
criterion = nn.CrossEntropyLoss()
logits = model(x)
loss = criterion(logits, y)          # y is a single integer: the gold relation ID
preds = torch.argmax(logits, dim=1)
correct += (preds == y).sum().item()
```

TRAINING LOOP MECHANICS: 1. `logits = model(x)`: shape $[\text{batch}, 916]$ 2. `criterion(logits, y)`: internally applies Softmax, then computes $-\log(P(\text{true_class}))$ 3. `loss.backward()`: PyTorch computes gradient

of loss w.r.t. ALL parameters via chain rule 4. `optimizer.step()`: updates ALL parameters by subtracting `learning_rate * gradient`

WHAT WAS THE ACTUAL BOTTLENECK? This model predicts ONE relation from 916, for ONE hop. But CWQ has questions with 2, 3, or 4 hops. This model simply cannot handle multi-hop. For a 2-hop question, the model's single output was treated as "the path." If hop 1 was correct: accuracy was measured as correct. If hop 2 also needed to be correct: the model has NO mechanism for it.

Result: Dev Hits@1 = 32.53%

EARLY STOPPING:

```
if dev_hit1 > best_dev_hit1:
    best_dev_hit1 = dev_hit1
    patience_counter = 0
    torch.save(model.state_dict(), "relation_flat_best.pt")
else:
    patience_counter += 1
if patience_counter >= patience=3:
    break
```

If validation accuracy doesn't improve for 3 consecutive epochs: stop training. Saves the checkpoint at the BEST point (not the last). `model.state_dict()`: a dictionary of all parameter tensors → saved to disk.

4.2 FILE: train/exp1_domain_baseline.py — EXPERIMENT 1

PURPOSE: Learn to predict the DOMAIN (topic area) of the question, NOT the specific relation. WHY: Knowing the domain lets us FILTER from 916 relations to ~30 in that domain.

```
class BERTDomainClassifier(nn.Module):
    def __init__(self, encoder_model="bert-base-uncased", hidden_dim=1024,
num_domains=69):
        self.encoder = QuestionEncoder(model_name=encoder_model)    # BERT is now
LIVE
        self.net = nn.Sequential(
            nn.Linear(self.encoder.output_dim, hidden_dim),
            nn.GELU(),
            nn.LayerNorm(hidden_dim),
            nn.Linear(hidden_dim, num_domains)    # Output: 69 domain logits
        )
```

CRITICAL DIFFERENCE from Exp 0: Exp 0: BERT was run OFFLINE, embeddings saved to disk, MLP trained separately. Exp 1: BERT is run INLINE during training (LIVE bert inference every batch). This means the BERT weights are also updated via backpropagation. ("Fine-tuning BERT") The gradient flows from the domain classification loss all the way back into BERT.


```
def collate_fn(batch, tokenizer):
    questions = [item[0] for item in batch]
    targets = torch.tensor([item[1] for item in batch], dtype=torch.long)
    encoded = tokenizer(questions, padding=True, truncation=True,
                        max_length=128, return_tensors="pt")
    return encoded, targets
```

`tokenizer(questions, padding=True, truncation=True)`: `padding=True`: pad all questions in batch to same length with [PAD] tokens. `truncation=True`: cut any questions longer than 128 tokens (very rare in CWQ). `max_length=128`: maximum sequence length. `return_tensors="pt"`: return PyTorch tensors, not Python lists.

`targets = torch.tensor([item[1] for item in batch])`: Integer domain IDs. Example: [14, 7, 14, 42, 7] for a batch of 5 questions.

```
num_domains = int(torch.max(train_d).item()) + 1
model = BERTDomainClassifier(num_domains=num_domains)
```

We dynamically determine `num_domains` from the data (max domain ID + 1). This makes the code handle datasets with different numbers of domains automatically.

```
optimizer = torch.optim.AdamW(model.parameters(), lr=2e-5)
```

AdamW: Adam optimizer with Weight Decay. Adam: Adaptive learning rates per parameter. Uses first moment (mean gradient) and second moment (mean squared gradient) to scale the learning rate. Weight Decay: An L2 regularization penalty added to the update rule. Prevents weights from growing too large (overfitting). `lr=2e-5`: 0.00002. Deliberately tiny because we're fine-tuning a pre-trained BERT. Too large and we'd "forget" the pre-trained knowledge.

```
scaler = torch.amp.GradScaler('cuda')
with torch.amp.autocast('cuda'):
    logits = model(input_ids, attention_mask)
    loss = criterion(logits, y)
scaler.scale(loss).backward()
scaler.step(optimizer)
scaler.update()
```

MIXED PRECISION TRAINING: `autocast`: runs the FORWARD PASS in float16 (2 bytes vs 4 bytes). - Halves VRAM usage: fits larger batches - 2× speedup on Tensor Cores (special hardware for float16 matmul) `GradScaler`: loss is multiplied by a large scaling factor before `backward()`. - float16 has limited precision range. Very small gradients become exactly 0.0. - Scaling makes small gradients large enough to survive float16 representation. - After optimizer step, the scale factor is adjusted up/down dynamically.

THE DOMAIN-RESTRICTED SEARCH (Exp 1's runtime behavior):

```

class DomainRestrictedSearcher:
    def search(self, question_text, start_entity, beam_width=3):
        logits = self.model(inputs)
        top_domains = torch.topk(logits, k=beam_width).indices[0].tolist()
        allowed_relations = {r_idx for r_idx, d_idx in
enumerate(relation_to_domain)
                        if d_idx in top_domains}
        beam = [(start_entity, [])]
        for hop in range(2):
            new_beam = []
            for entity, path in beam:
                neighbors = self.kg.get_neighbors(entity)
                for r, t, direction in neighbors:
                    if r in allowed_relations: # DOMAIN FILTER
                        new_beam.append((t, path + [(r, t)]))
            beam = new_beam[:beam_width]
        return beam

```

We take top-3 domains (not just top-1) to HEDGE against wrong domain prediction. At test time, if the model predicts ["people", "film", "music"] as the top-3 domains, only relations from these 3 domains (~90 relations) are considered. The OTHER ~826 relations are completely ignored.

ACCURACY: Dev Hits@1 = ~30%. Domain filtering helped beam search but hurt relation accuracy.

4.3 FILE: train/exp2_cpd.py — EXPERIMENT 2: Contrastive Path Decoding v1

PURPOSE: Learn a SHARED EMBEDDING SPACE where similar questions are close to their correct paths. This is a "Metric Learning" approach, not a "Classification" approach.

```

class CPDModel(nn.Module):
    def __init__(self):
        self.q_encoder = QuestionEncoder() # Question → 768-dim
        self.rel_embed = nn.Embedding(num_relations, 128) # Relation lookup
table
        self.p_encoder = PathEncoder(128, 256) # Path LSTM → 512-dim
        self.q_proj = nn.Linear(768, 512) # Question 768 → 512
        self.temperature = 0.07 # InfoNCE temperature

```

TWO ENCODERS: q_encoder: encodes the QUESTION into 512-dim (after projection) p_encoder: encodes the RELATION PATH into 512-dim (via LSTM)

They share the same 512-dim space so we can compute cosine similarity between them.

```

def forward(self, input_ids, attention_mask, pos_path_ids):
    q_repr = F.normalize(self.q_proj(q_h), p=2, dim=-1) # Unit-normed question
vector

```

```

    p_repr = self.encode_path(pos_path_ids)          # Unit-normed path
vector
    # Build HARD NEGATIVES via relation similarity matrix
    sim_matrix = torch.matmul(rel_norm, rel_norm.T)    # [916, 916]
similarity
    for i in range(B):
        for _ in range(4):                            # 4 negatives per
sample
            swap_idx = random_hop_position
            orig_rel = path[swap_idx]
            topk = torch.topk(sim_matrix[orig_rel], k=6) # Most similar
relations
            swap_rel = topk[topk != orig_rel][0]        # Pick best similar ≠
original
            new_path = path.clone()
            new_path[swap_idx] = swap_rel                # Swap one hop

```

HARD NEGATIVE CONSTRUCTION: Given gold path: [film.film.actor, people.person.nationality] Compute which relation is most similar to "film.film.actor": → "film.film.director" (also about films) has high score Create negative: [film.film.director, people.person.nationality] This negative LOOKS almost like the correct path but is WRONG. Training against these hard negatives makes the model DISCRIMINATING.

Why hard negatives beat random negatives:
Random negative: [chemistry.element.symbol, food.ingredient.cuisine]
→ Completely different domain, easy to reject, model learns nothing hard
Hard negative: [film.film.director, people.person.nationality]
→ Same domain, similar words, model must learn the subtle difference

```

pos_score = torch.sum(q_repr * p_repr, dim=-1) / 0.07    # Cosine sim /
temperature
neg_scores = torch.sum(q_repr.unsqueeze(1) * neg_repr, dim=-1) / 0.07
logits = torch.cat([pos_score.unsqueeze(1), neg_scores], dim=1)
target = torch.zeros(B, dtype=torch.long)                # Target = index 0 =
positive
loss = F.cross_entropy(logits, target)

```

InfoNCE LOSS STEP BY STEP: pos_score: scalar similarity between question and correct path. E.g.: 8.5
neg_scores: [question, neg_path1], [question, neg_path2], ... E.g.: [7.2, 6.8, 7.0, 7.5] logits = [8.5, 7.2, 6.8, 7.0, 7.5]
(5 values for 1 positive + 4 negatives) target = 0 (we want the model to say "index 0 = the positive — is the best match") cross_entropy(logits, target=0) = -log(softmax([8.5, 7.2, 6.8, 7.0, 7.5])[0])

If pos_score >> all neg_scores: softmax[0] ≈ 1.0, loss ≈ 0 (perfect)
If pos_score ≈ neg_scores: softmax[0] ≈ 0.2, loss = -log(0.2) ≈ 1.6 (poor)

The temperature $\tau=0.07$ makes the distribution SHARPER:

Without τ : logits [8.5, 7.2] $\rightarrow$ softmax [0.79, 0.21] $\rightarrow$ relatively easy
 With $\tau=0.07$: logits [8.5/0.07, 7.2/0.07] = [121, 102] $\rightarrow$ softmax $\approx$ [1.0, 0.0] $\rightarrow$ much harder

4.4 FILE: train/exp3_pct.py — EXPERIMENT 3: Path Confidence Tracker

PURPOSE: Multi-task learning with domain, subdomain, relation, AND confidence prediction. Novel idea: the model learns to estimate "was I right?" alongside predicting the answer.

```
class PCTModel(nn.Module):
    def __init__(self):
        self.shared_net = nn.Sequential(
            nn.Linear(768, 1024), nn.GELU(), nn.LayerNorm(1024)
        )
        self.domain_head = nn.Linear(1024, num_domains)      # What topic area?
        self.subdomain_head = nn.Linear(1024, 200)           # Finer-grained topic
        (mocked)
        self.relation_head = nn.Linear(1024, num_relations)  # What relation?
        self.confidence_head = nn.Linear(1024, 1)            # Am I sure?
```

MULTI-TASK ARCHITECTURE: ONE trunk (shared_net) extracts features from the question. FOUR heads, each specializing in a different prediction task. Gradients from ALL four tasks flow back through the shared trunk. This forces the trunk to learn representations useful for ALL tasks.

Insight: Relations and Domains are related. A model that understands domains better also predicts relations better. Sharing computation is efficient.

`subdomain_head` with `num_subdomains=200`: The CWQ relation vocab has ~916 relations but they don't have neat "subdomains." We MOCKED subdomains as: `self.subdomains = self.relations % 200` This is a hack — assigning pseudo-subdomains by modulo operation. The subdomain head forced the model to learn finer-grained features even though the targets were artificial. In practice it added regularization.

```
with torch.no_grad():
    preds = torch.argmax(rel_logits, dim=-1)
    is_correct = (preds == r).float()      # 1.0 if right, 0.0 if wrong
    loss_c = bce_loss(conf_logits, is_correct)
```

THE CONFIDENCE HEAD TRAINING TRICK: We DON'T need to know in advance if the model is right. We COMPUTE whether it was right (`argmax == gold`) DURING training. Then we train the confidence head to predict that 0/1 outcome.

The ``torch.no_grad()`` wrapper means: compute ``is_correct`` without tracking gradients.
 We don't want the gradient of ``is_correct`` to flow back — we're using it only as a

label.

Intuition: The model has a "gut feeling" (confidence logit) before seeing the answer.

We train this gut feeling to match reality. After training: if the confidence is high, the model's relation prediction is likely correct. If low, the model might be guessing.

```
loss = loss_d + loss_s + loss_r + loss_c
loss.backward()
```

ALL FOUR LOSSES ARE SUMMED without any weighting. Each loss contributes equally to the gradient that updates the shared trunk. This is a simplification; in practice, loss weighting could be tuned.

RESULT: Dev Hits@1 = 30.85% — slightly WORSE than Exp 0. WHY? Adding more tasks hurt the single-hop relation prediction. The model's "thinking resources" were spread across 4 tasks instead of 1. The architecture wasn't deep enough to handle all tasks simultaneously.

LESSON: Multi-task helps when tasks are CLOSELY related and the model is large enough. With only a 2-layer MLP trunk, the added complexity hurt more than it helped.

4.5 FILE: train/exp4_chcp.py — EXPERIMENT 4: Cross-Hop Coherence Planning

THE BREAKTHROUGH EXPERIMENT. The jump from 32% to 56% happened here. Two key innovations: (1) Hop Embeddings and (2) Coherence Loss.

```
class CHCPModel(nn.Module):
    def __init__(self):
        self.q_encoder = QuestionEncoder()
        self.proj = nn.Linear(768, 256)
        self.hop_embeddings = nn.Parameter(torch.randn(4, 256)) # LEARNABLE HOP
POSITIONS
        encoder_layer = TransformerEncoderLayer(d_model=256, nhead=4,
batch_first=True)
        self.transformer = TransformerEncoder(encoder_layer, num_layers=2) #
CROSS-HOP REASONING
        self.relation_head = nn.Linear(256, num_relations) # PREDICTS
RELATION PER HOP
        self.stop_head = nn.Linear(256, 1) # PREDICTS
STOP PER HOP
        self.transition_matrix = nn.Parameter(torch.randn(916, 916)) # COHERENCE
MATRIX
```

HOP EMBEDDINGS IN DETAIL: `self.hop_embeddings = nn.Parameter(torch.randn(4, 256))`: - `nn.Parameter`: marks this tensor as a TRAINABLE PARAMETER. Unlike regular tensors, Parameters are

included in `model.parameters()` and updated by the optimizer. They are saved in `state_dict()`. - Initially random (`torch.randn`). After training: encodes structural knowledge. - `hop_embeddings[0]`: "What does a FIRST-hop question look like?" - `hop_embeddings[1]`: "What does a SECOND-hop question look like?" - etc.

After training the model, examining `hop_embeddings`:

- `hop_embeddings[0]` will converge to represent "entity → attribute" patterns (1st hop is usually about finding a related entity from the starting one)
- `hop_embeddings[3]` will represent more "specific attribute" patterns (4th hop is usually a final attribute like nationality, date, etc.)

```
def forward(self, input_ids, attention_mask):
    q_h = self.q_encoder(input_ids, attention_mask) # [B, 768]
    q_proj = self.proj(q_h) # [B, 256]
    init_repr = q_proj.unsqueeze(1) + self.hop_embeddings.unsqueeze(0) # [B, 4, 256]
```

BROADCASTING ADDITION: `q_proj.unsqueeze(1)`: [B, 256] → [B, 1, 256] `hop_embeddings.unsqueeze(0)`: [4, 256] → [1, 4, 256] Addition with broadcasting: [B, 4, 256]

What does this produce?

```
init_repr[b, 0, :] = question_vector + hop0_embedding ("1st-hop version of this question")
init_repr[b, 1, :] = question_vector + hop1_embedding ("2nd-hop version of this question")
init_repr[b, 2, :] = question_vector + hop2_embedding ("3rd-hop version of this question")
init_repr[b, 3, :] = question_vector + hop3_embedding ("4th-hop version of this question")
```

Same question, 4 slightly different perspectives – one for each reasoning step.

```
refined_repr = self.transformer(init_repr) # [B, 4, 256]
```

TRANSFORMER ENCODER (Cross-Hop Reasoning): The Transformer takes the 4 hop representations as a SEQUENCE. It performs multi-head self-attention: each hop representation "looks at" all others.

Concretely, the attention mechanism for hop 2 computes:

$$\text{attention}(\text{hop2}, \text{all_hops}) = \text{softmax}(\text{Q}_2 * \text{K}_{\text{all}}^T / \sqrt{d}) * \text{V}_{\text{all}}$$

Where Q, K, V are learned linear projections of hop representations.

This means hop 2's final representation is influenced by what hop 1 and hop 3 look like.

If the model predicts hop 1 = "film.film.actor", hop 2's attention sees this and

adjusts its prediction TOWARDS nationality/birthplace (things about people in film).

This is why this experiment is called "CROSS-HOP" – hops influence each other. All 4 hops are computed in ONE JOINT FORWARD PASS.

```
rel_logits = self.relation_head(refined_repr) # [B, 4, 916]
stop_logits = self.stop_head(refined_repr).squeeze(-1) # [B, 4]
```

OUTPUT STRUCTURE: $\text{rel_logits}[b, h, r]$ = how strongly model believes relation r is correct at hop h for sample b . $\text{stop_logits}[b, h]$ = should we stop reasoning after completing hop h ?

RELATION LOSS:

```
ce_loss = nn.CrossEntropyLoss(ignore_index=0) # 0 is the PAD relation
loss_r = ce_loss(rel_logits.view(-1, num_relations), paths.view(-1))
```

$\text{rel_logits.view}(-1, 916)$: reshapes $[B, 4, 916] \rightarrow [B4, 916]$. All hops treated as one batch.

$\text{paths.view}(-1)$: reshapes $[B, 4] \rightarrow [B4]$. 4 targets (one per hop) per sample. $\text{ignore_index}=0$: When the gold path is $[312, 456, 0, 0]$ (2-hop, padded with 0), the loss for hops 3 and 4 (where gold = 0) is NOT computed. This prevents the model from being penalized for hop 3 of a 2-hop question.

STOP LOSS:

```
stop_targets = (paths == 0).float() # 1.0 where gold is PAD (= should stop), 0.0 otherwise
loss_stop = bce_loss(stop_logits, stop_targets)
```

Gold path $[312, 456, 0, 0]$: $\text{stop_targets} = [0.0, 0.0, 1.0, 1.0]$ Interpretation: "Don't stop at hop1, don't stop at hop2, stop at hop3 and hop4"

BCEWithLogitsLoss: For each of the 4 hops: $\text{loss} = -[\text{target} * \log(\text{sigmoid}(\text{logit})) + (1-\text{target}) * \log(1-\text{sigmoid}(\text{logit}))]$ If $\text{target}=1$ and $\text{sigmoid}(\text{logit}) = 0.9$: $\text{loss} = -\log(0.9) \approx 0.1$ (good) If $\text{target}=0$ and $\text{sigmoid}(\text{logit}) = 0.1$: $\text{loss} = -\log(0.9) \approx 0.1$ (good) If wrong: loss can be large

COHERENCE LOSS:

```
self.transition_matrix = nn.Parameter(torch.randn(916, 916))

trans_probs = F.log_softmax(model.transition_matrix, dim=-1) # [916, 916] of log probs
for k in range(max_hops - 1):
    r_k = preds[:, k] # Predicted relation at hop k (integer IDs) [B]
    r_k1 = preds[:, k+1] # Predicted relation at hop k+1 (integer IDs) [B]
```

```

valid_mask = (r_k != 0) & (r_k1 != 0) # Only consider non-padding hops
if valid_mask.any():
    log_p = trans_probs[r_k[valid_mask], r_k1[valid_mask]] # Look up
transition prob
    coherence_loss -= log_p.mean() # MAXIMIZE log probability (minimize
negative)

```

TRANSITION MATRIX MECHANICS: `transition_matrix[i][j]`: raw logit for "relation j should follow relation i" After `log_softmax(dim=-1)`: row i gives the log-probability distribution over which relation should come NEXT given that relation i was used at this hop.

During training:

- If the model predicts "film.film.actor" → "people.person.nationality": this pair appears frequently in data, so `trans_probs[film.actor, person.nationality]` becomes HIGH.
- If "chemistry.element" → "film.actor" appears NEVER: `trans_probs[chem, film]` stays LOW.

Coherence Loss: `-log_p.mean()` = average negative log-probability of all consecutive pairs.

Minimizing this loss = maximizing the log-probability = making transitions MORE consistent

with Freebase's natural relation sequencing patterns.

Weighted at `0.1 * coherence_loss` – small to avoid dominating the relation loss.

OVERALL LOSS: `total = loss_r + loss_stop + 0.1 * coherence_loss`

RESULT: Dev Hits@1 = 56.20%. A 24% jump from Exp 0! WHY DID IT HELP? 1. Transformer allows hops to "talk to each other" — planning is JOINT not independent. 2. Hop embeddings give position-awareness — model knows it's at step 1 vs step 3. 3. Coherence loss enforces relation sequences to follow Freebase's logical flow.

4.6 FILE: train/exp5_rlmc.py — EXPERIMENT 5: RL Blueprint

This file is an ARCHITECTURAL PROTOTYPE, not a full training script. The `env.step()` function uses `torch.randn` for state — it's a mock. The actual PPO logic here was the BLUEPRINT that became Exp 9.

```

NUM_ACTIONS = 4
# Action 0: TIGHT (top-1 relation)
# Action 1: MEDIUM (top-5 relations)
# Action 2: LOOSE (domain fallback - all relations in domain)
# Action 3: STOP

class RLMPolicy(nn.Module):
    def __init__(self):
        self.net = nn.Sequential(Linear(state_dim, hidden), GELU, LayerNorm,
Linear(hidden, hidden))
        self.actor = nn.Sequential(nn.Linear(hidden, 4)) # 4 action logits
        self.critic = nn.Sequential(nn.Linear(hidden, 1)) # Value estimate

```



```
def forward(self, state):
    h = self.net(state)
    probs = F.softmax(self.actor(h), dim=-1)    # Action probabilities
    value = self.critic(h).squeeze(-1)         # Scalar value estimate
    return probs, value, logits
```

STATE SPACE DEFINITION: state_dim = question_dim(768) + entity_dim(128) + path_dim(128) + 2 The +2 is for: [hop_number (scalar), candidate_count (scalar)]

```
State = [
    768-dim: "What is the question asking?"
    128-dim: "What entity are we currently at?"
    128-dim: "What path have we taken so far?"
    1-dim:   "Which hop number is this?"
    1-dim:   "How many entities are in our current set?"
]
```

The candidate_count is CRUCIAL: if we're at hop 2 and have 500 candidate entities, that means the previous action was too loose. The agent learns: "high count → go TIGHT."

```
def compute_ppo_loss(old_probs, new_probs, advantages, returns, values,
epsilon=0.2):
    ratio = new_probs / (old_probs + 1e-8)
    surr1 = ratio * advantages
    surr2 = torch.clamp(ratio, 1.0 - epsilon, 1.0 + epsilon) * advantages
    actor_loss = -torch.min(surr1, surr2).mean()    # The "Proximal" part
    critic_loss = F.mse_loss(values, returns)
    entropy = -(new_probs * torch.log(new_probs + 1e-8)).sum(-1).mean()
    loss = actor_loss + 0.5 * critic_loss - 0.01 * entropy
    return loss
```

PPO CLIPPING (The "Proximal" in PPO): $\text{ratio} = \text{new_probs} / \text{old_probs}$: How much has the policy changed for the action that was taken? If ratio > 1: the new policy takes this action MORE often than before. If ratio < 1: the new policy takes it LESS often.

``torch.clamp(ratio, 1-ε, 1+ε)`` with $\epsilon=0.2$:

If ratio > 1.2: clip to 1.2 (don't increase probability too much in one step)

If ratio < 0.8: clip to 0.8 (don't decrease probability too much in one step)

WHY? Without clipping, a single good rollout could push the policy too far.

This causes the policy to "overfit" to one batch of experience and forget others.

Clipping keeps updates CONSERVATIVE and STABLE.

``surr1 = ratio * advantages``: Unconstrained update

``surr2 = clamped_ratio * advantages``: Constrained update

```
`min(surr1, surr2)` : Take the MORE CONSERVATIVE of the two.
    If advantage > 0 (good action): surr1 > surr2 when ratio > 1+ε. Use surr2
    (clamped).
    If advantage < 0 (bad action): surr2 > surr1 when ratio < 1-ε. Use surr1.
    In both cases, we prevent overshooting.
```

4.7 FILE: train/exp6_unified.py — EXPERIMENT 6: Unified Adaptive Planner

PURPOSE: Combine the best of Exp 3 (domain awareness) and Exp 4 (cross-hop planning).

```
class UnifiedKGQAPlanner(nn.Module):
    def __init__(self):
        # === FROM EXP 3 ===
        self.domain_head = nn.Linear(hidden_dim, num_domains)
        self.confidence_head = nn.Linear(hidden_dim, 1)
        # === FROM EXP 4 ===
        self.hop_embeddings = nn.Parameter(torch.randn(4, 256))
        self.transformer = nn.TransformerEncoder(...)
        self.relation_head = nn.Linear(hidden_dim, num_relations)
        self.adaptive_stop_head = nn.Linear(hidden_dim, 1)
```

NEW: AUTOMATED DOMAIN EXTRACTION:

```
main_rel = path[0][1] # First relation in path
domain = main_rel.split('.')[0] if '.' in main_rel else 'none'
```

Domain is extracted DIRECTLY from the gold relation string at DATA LOADING time. No separate annotation needed. "people.person.nationality" → split(".")[0] → "people" → domain2id["people"] = 14

COMBINED LOSS:

```
loss_dom = F.cross_entropy(out['domain_logits'], doms) # 69 domains
loss_rel = F.cross_entropy(out['rel_logits'].view(-1, num_rel), paths.view(-1)) #
916 relations
# Stop targets: 1.0 for real hops, 0.0 for padding
stop_targets[b, :nums[b]] = 1.0
loss_stop = F.binary_cross_entropy_with_logits(out['stop_logits'], stop_targets)
total_loss = loss_dom + loss_rel + loss_stop
```

NOTE: Coherence loss from Exp 4 was DROPPED here to simplify the combined training. The domain loss compensates for some of this by providing high-level structural guidance.

RESULT: Dev Hits@1 = 53.60% — SLIGHTLY WORSE than Exp 4 alone (56.20%). WHY? Adding domain loss diluted the gradient signal for relation prediction. The model spread its capacity across the extra domain

classification task. This taught us: when architecture improvements conflict, need deeper investigation.

4.8 FILE: train/exp7_roberta.py — EXPERIMENT 7: Scaling to RoBERTa-Large

PURPOSE: Scale the Exp 6 architecture with a much larger, better pre-trained language model.

```
class ScaledUnifiedPlanner(nn.Module):
    def __init__(self):
        self.encoder = RobertaModel.from_pretrained("roberta-large")
        self.encoder_dim = self.encoder.config.hidden_size # 1024 (vs BERT's 768)
        self.proj = nn.Linear(1024, 512)
        encoder_layer = TransformerEncoderLayer(d_model=512, nhead=8) # 8 heads
        (was 4)
        self.transformer = TransformerEncoder(encoder_layer, num_layers=4) # 4
        layers (was 2)
```

RoBERTa vs BERT — Key Structural Differences: RoBERTa-Large: 355 million parameters (vs BERT-base: 110M)
 Hidden size: 1024 (vs 768) Layers: 24 (vs 12) Attention heads: 16 (vs 12) Training data: 160GB (vs 16GB)
 Training improvements: - No "Next Sentence Prediction" task (removes a flawed pre-training objective) -
 Dynamic masking: different tokens masked each epoch (not static) - Larger batches: 8000 sequences (vs 256) -
 More steps: 500K (vs 1M but with smaller batches)

```
q_h = outputs.last_hidden_state[:, 0, :] # CLS token, directly from last layer
```

In Exp 1 we used `outputs.pooler_output` (CLS passed through extra linear + tanh). In Exp 7 we use `last_hidden_state[:, 0, :]` directly. For RoBERTa, the pooler is less well-calibrated (not used in original pre-training). Direct CLS extraction gives slightly better representation quality.

GRADIENT ACCUMULATION (Critical for Memory):

```
train_loader = DataLoader(train_ds, batch_size=4) # Only 4 samples per batch!
accumulation_steps = 4

for i, batch in enumerate(train_loader):
    total_loss = compute_loss(batch) / accumulation_steps # DIVIDE by 4
    scaler.scale(total_loss).backward() # Accumulate gradients
    if (i + 1) % accumulation_steps == 0: # Every 4th step:
        scaler.step(optimizer) # ACTUALLY update
weights
    scaler.update()
    optimizer.zero_grad()
```

WITHOUT gradient accumulation (effective batch_size=4): - Each optimizer step sees only 4 samples - Training is very noisy (large variance in gradient direction) - Model oscillates; takes much longer to converge

WITH gradient accumulation (effective batch_size=16): - Gradients from 4 micro-batches of 4 are SUMMED before updating weights - Each optimizer step sees 16 samples worth of gradient - Smoother gradient, stable convergence - Memory usage stays at batch_size=4 levels (only 4 samples in GPU at once)

WHY divide by accumulation_steps?

If we don't divide: after accumulating 4 micro-batches, the total gradient is 4× what it should be for a true batch of 16.

Dividing ensures the effective gradient magnitude matches a real batch_size=16.

RESULT: Dev Hits@1 = 56.40%. Small improvement over Exp 4's 56.20%. The bigger model helped but not dramatically — the architecture bottleneck was elsewhere.

4.9 FILE: train/exp8_cpd_roberta.py — EXPERIMENT 8: Contrastive RoBERTa

PURPOSE: Apply DYNAMIC HARD NEGATIVE contrastive learning on top of the Exp 7 model.

```
from train.exp7_roberta import ScaledUnifiedPlanner
model = ScaledUnifiedPlanner(num_dom, num_rel, hidden_dim=512).to(device)
# Load Exp 7 weights as starting point
model.load_state_dict(torch.load('checkpoints/exp7_roberta_best.pt'))
optimizer = torch.optim.AdamW(model.parameters(), lr=5e-6) # Much smaller LR!
```

WHY 5e-6 vs 1e-5 in Exp 7? We are FINE-TUNING an already-good model, not training from scratch. Large updates would "destroy" the good representations learned in Exp 7. Tiny LR makes small, surgical adjustments to decision boundaries.

```
def path_contrastive_loss(logits, gold_paths, lens, tau=0.1):
    for b in range(B):
        L = int(lens[b].item()) # True number of hops for
        this sample
        pos_score = sum([logits[b, h, gold_paths[b, h]] for h in range(L)])
```

pos_score: For a 2-hop question with gold path [312, 456]: pos_score = logits[b, 0, 312] + logits[b, 1, 456]
This is the SUM of the log-likelihood the model assigns to EACH CORRECT relation. A higher pos_score means the model is more confident about the full path.

```
for h in range(L):
    hop_logits = logits[b, h] # All 916 logit values at
    this hop
    top2 = torch.topk(hop_logits, 2).indices # Indices of top-2 highest
    logits
    neg_r = top2[1] if top2[0] == gold_paths[b, h] else top2[0]
```

DYNAMIC HARD NEGATIVE SELECTION: At hop h , look at what the model is CURRENTLY most confident about. If top-1 is correct (= gold): use top-2 as the wrong alternative (second-best mistake) If top-1 is wrong (not gold): use top-1 as the wrong alternative (model's biggest mistake)

This negative "follows" the model. As training progresses and the model gets better, the negatives automatically become harder because the model's top predictions improve.

```
n_score = pos_score - hop_logits[gold_paths[b,h]] + hop_logits[neg_r]
```

n_score = score of the ADVERSARIAL path (one hop swapped): Start with pos_score (sum of gold logits for all hops) Remove the gold logit at hop h : $- \text{logits}[b, h, \text{gold_at_h}]$ Add the adversarial logit at hop h : $+ \text{logits}[b, h, \text{neg_r}]$ Result: score of a path identical to gold EXCEPT at hop h , where we use neg_r .

This creates L different near-perfect-but-wrong paths for an L -hop question.

```
all_scores = torch.stack([pos_score] + neg_scores) / tau
loss_b = -F.log_softmax(all_scores, dim=0)[0] # Index 0 = positive must
win
```

Final loss for this sample: the positive must have the HIGHEST score. Temperature $\tau=0.1$: sharper than $\tau=0.07$ used in Exp 2. The model is already pre-trained so we need a harder contrastive target.

TOTAL LOSS in Exp 8:

```
total_loss = (loss_dom + loss_rel + loss_stop + lambda_cpd * loss_cpd) /
accumulation_steps
```

$\lambda_{cpd} = 0.5$: the contrastive loss is weighted at half the standard losses.

RESULT: Dev Hits@1 = 58.20% — the highest non-RL score. The dynamic hard negatives successfully tightened the decision boundaries. The model learned to be MORE CERTAIN about the correct path vs. near-miss alternatives.

4.10 FILE: train/exp9_rlmc.py — EXPERIMENT 9: RLMC — THE STATE OF THE ART

RESULT: Test Hits@1 = 76.66% (compared to previous SOTA DRKG = 66.9%)

```

class RLConstraintAgent(nn.Module):
    def __init__(self, base_model):
        self.base_model = base_model
        for param in self.base_model.parameters():
            param.requires_grad = False # FREEZE Exp 7/8 backbone completely
        self.policy_head = nn.Sequential(
            nn.Linear(512, 256), nn.ReLU(), nn.Linear(256, 4) # 4 action outputs
        )
        self.value_head = nn.Sequential(
            nn.Linear(512, 256), nn.ReLU(), nn.Linear(256, 1) # 1 value output
        )

```

TWO-PHASE TRAINING PHILOSOPHY: Phase 1 (Exp 7/8): Train the "Brain" (RoBERTa planner) on supervised path prediction. The brain learns which relations exist and how they sequence. Phase 2 (Exp 9): Freeze the Brain. Train only a small "Decision Head" (policy_head) to learn HOW WIDE to search at each hop.

WHY FREEZE? We DON'T want RL to change the brain's relation predictions. The brain is already good at ranking relations. RL just needs to decide: "Given the brain's confidences, how many of the top relations should I explore?"

The policy_head has only ~100K parameters (vs Brain's 355M).
RL training is MUCH faster on a tiny model.

```

def forward(self, input_ids, attention_mask):
    with torch.no_grad(): # No gradients for brain!
        outputs = self.base_model.encoder(input_ids, attention_mask)
        q_h = outputs.last_hidden_state[:, 0, :]
        h_q = self.base_model.proj(q_h)
        init_repr = h_q.unsqueeze(1) + self.base_model.hop_embeddings.unsqueeze(0)
        refined_repr = self.base_model.transformer(init_repr)
        rel_logits = self.base_model.relation_head(refined_repr) # [B, 4, 916]
        domain_logits = self.base_model.domain_head(h_q) # [B, 69]
        action_logits = self.policy_head(refined_repr) # [B, 4, 4]
        state_values = self.value_head(refined_repr).squeeze(-1) # [B, 4]
        return action_logits, state_values, rel_logits, domain_logits

```

FLOW: 1. Brain processes question (frozen, no grad) 2. Brain produces refined_repr [B, 4, 512]: the per-hop representations 3. Policy head takes these representations and outputs action logits [B, 4, 4] (4 actions at each of 4 hops) 4. Value head estimates total expected reward for each hop [B, 4]

REWARD CALCULATION:

```

def calculate_meta_rewards(actions, rel_logits, domain_logits, gold_paths,
                           gold_domains, path_lengths):

```

```

for b in range(B):
    L = path_lengths[b]    # True number of hops (1-4)
    for h in range(max_hops):
        a = actions[b, h]
        if h >= L:         # PAST THE END OF THE PATH
            rewards[b, h] = +1.0 if a == 3 else -1.0    # STOP? Good. Otherwise
bad.
        else:
            gold_r = gold_paths[b, h]
            logits_h = rel_logits[b, h]
            if a == 0: # TIGHT: top-1 only
                top1 = argmax(logits_h)
                rewards[b, h] = +1.0 if top1 == gold_r else -1.0
            elif a == 1: # MEDIUM: top-5
                top5 = topk(5)
                rewards[b, h] = +0.5 if gold_r in top5 else -1.0
            elif a == 2: # LOOSE: domain
                pred_dom = argmax(domain_logits[b])
                rewards[b, h] = +0.1 if pred_dom == gold_dom else -1.0
            elif a == 3: # STOP too early
                rewards[b, h] = -1.0

```

REWARD DESIGN INSIGHT: TIGHT correct = +1.0: Maximum reward. Being right AND efficient. MEDIUM correct = +0.5: Half reward. Right but explored 5× more. LOOSE correct = +0.1: Tiny reward. Right but explored 50× more. Any incorrect = -1.0: Heavy penalty for dead ends / wrong answers. Early STOP = -1.0: Same penalty — stopping before getting the answer.

The AGENT LEARNS: "When the model's top-1 is almost certainly correct (high gap between top-1 and top-2 logits), use TIGHT. When the logits are close together (uncertain), use MEDIUM or LOOSE to hedge."

PPO TRAINING LOOP:

```

probs = F.softmax(action_logits, dim=-1)                # [B, 4, 4]
m = torch.distributions.Categorical(probs)               # Define probability
distribution
actions = m.sample()                                    # Sample one action per
(batch, hop)
log_probs = m.log_prob(actions)                         # Log probability of chosen
actions

rewards = calculate_meta_rewards(actions, ...)

# Discounted returns via Bellman equation
G = 0
for h in reversed(range(H)):
    G = rewards[b, h] + gamma=0.99 * G
    returns[b, h] = G
    adv[b, h] = G - state_values[b, h]    # Advantage = actual - expected

```

```

actor_loss = -(log_probs * adv).mean()           # Policy gradient loss
critic_loss = F.mse_loss(state_values, returns)   # Value function loss
entropy_bonus = -m.entropy().mean() * 0.01       # Exploration bonus
loss = actor_loss + 0.5 * critic_loss + entropy_bonus

```

DISCOUNTED RETURNS (Bellman): $G = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots$ $\gamma=0.99$: future rewards count 99% as much as immediate ones. Computed BACKWARD (from last hop to first) for efficiency.

If hop3 has reward +1, hop4 has reward +1 (STOP):

$G(\text{hop4}) = +1$

$G(\text{hop3}) = +1 + 0.99 * 1 = 1.99$

$G(\text{hop2}) = \dots$ (depends on hop3's reward)

The return at hop 2 INCLUDES the future reward of getting to the answer at hop3/4. This teaches the agent: "Choosing a wide action here (even if only +0.5) is good if it enables reaching the answer later."

ADVANTAGE: $\text{adv}[b, h] = G[b, h] - \text{state_values}[b, h]$ state_values is the CRITIC's estimate of G BEFORE taking the action. Advantage > 0: "This was better than expected → do it more often" Advantage < 0: "This was worse than expected → do it less often"

ACTOR LOSS: $-(\text{log_probs} * \text{adv}).\text{mean}()$ For high-advantage actions: log_probs is negative (probabilities < 1), so $-\text{log_prob} * \text{advantage} = \text{decrease loss} = \text{increase the action's probability}$. For negative-advantage actions: the opposite.

ENTROPY BONUS: $m.\text{entropy}()$ = $-\sum(p * \log(p))$ for each action distribution. Maximum entropy = 0.693 (uniform distribution = equal probability for all 4 actions). Minimum entropy = 0 (deterministic = probability 1 on one action). We SUBTRACT entropy from the loss (effectively ADD it to the reward). This PREVENTS the agent from becoming "addicted" to one action early in training. It forces continued EXPLORATION of all 4 action types.

4.11 FILE: train/exp10_universal.py (Currently Running)

PURPOSE: One universal model that handles CWQ, MetaQA, and WebQSP simultaneously.

DATASET ID EMBEDDING: `DATASET_IDS = {'cwq': 0, 'webqsp': 1, 'metaqa': 2}` The model gets a 3-dimensional "dataset ID" input telling it which dataset it's in. This is injected as an additional embedding added to the question representation.

TOPIC ENTITY INJECTION: "What nationality is Obama?" + topic_entity = "Barack Obama" → Input to model: "[CWQ] topic: Barack Obama | What nationality is Obama?" The entity name is concatenated directly into the input text. RoBERTa tokenizes and processes this as normal text.

SEQUENTIAL TRAINING PROTOCOL: Stage 1: CWQ (15 epochs) at $\text{lr}=2\text{e-}5$ — learn complex multi-hop Freebase reasoning Stage 2: WebQSP (5 epochs) at $\text{lr}=5\text{e-}6$ — adapt to real relation paths Stage 3: MetaQA (5 epochs) at $\text{lr}=5\text{e-}6$ — adapt to movie KG structure

Lower LR in stages 2/3: prevent "catastrophic forgetting" of CWQ knowledge.

SURGICAL HEAD EXPANSION (the fix we just implemented): When resuming from Exp 9 checkpoint (664 relations, 67 domains): - The new vocab has 861 relations, 70 domains. - We CANNOT directly load the old state_dict (size mismatch). - Solution: copy old weights into the NEW (larger) model, then load with strict=False.

```
ckpt_num_rel = state_dict['relation_head.weight'].shape[0] # 664
if ckpt_num_rel != num_rel: # num_rel = 861
    model.relation_head.weight.data[:ckpt_num_rel] =
state_dict['relation_head.weight']
    model.relation_head.bias.data[:ckpt_num_rel] =
state_dict['relation_head.bias']
    del state_dict['relation_head.weight'] # Remove to avoid size mismatch error
    del state_dict['relation_head.bias']
model.load_state_dict(state_dict, strict=False) # Load rest of weights
```

The new rows in the relation_head (indices 664 to 860) are RANDOMLY INITIALIZED. The model starts with some knowledge (old 664 relations) and learns the new 197 in-context.

PART 5: EVALUATION FILES

5.1 FILE: eval/e2e_evaluate.py — End-to-End Evaluator

This is the "Report Card" script. It loads EVERY trained model and evaluates all of them on both the dev set and test set, writing results to results.md.

EVALUATION PROTOCOL (two types):

Type A (Exp 0-8): PATH MATCHING Model predicts relation path → compare to gold path from SPARQL. Correct = predicted path exactly matches gold path. This is "planning accuracy" — it measures how well the model PLANS. Upper bounds actual answer accuracy (some wrong plans might still reach the answer by luck).

Type B (Exp 9): EXECUTION-BASED Model predicts path + RL actions → physically traverse KG → check if answer reached. Correct = the final set of entities contains at least one gold answer entity. This is directly comparable to published methods that use real Freebase execution.

```
def predict_exp0(model, tokenizer, question, device, num_rels, k=10):
    enc = tokenizer(question, ...)
    logits = model(enc['input_ids'], enc['attention_mask'])
    probs = F.softmax(logits, dim=-1)
    _, topk = torch.topk(probs, k=10, dim=-1)
    return topk[0].cpu().tolist() # Top 10 predicted relation IDs
```

For flat models (Exp 0, 3), the top-10 list is used to cover the multi-hop case: For a 2-hop gold path [312, 456]: Hop 1: is predictions[0] == 312? (Top-1 = first prediction) Hop 2: is predictions[1] == 456? (Second prediction used for second hop) Both must match for Hits@1.

```
def predict_exp4(model, tokenizer, question, device, max_hops=4, k=10):
    rel_logits, stop_logits = model(...)
    for h in range(max_hops):
        probs = F.softmax(rel_logits[0, h], dim=-1)
        _, topk = torch.topk(probs, k=10)
        stop_p = torch.sigmoid(stop_logits[0, h]).item()
        results.append({'top_ids': topk.tolist(), 'stop_prob': stop_p})
    return results
```

For hierarchical models (Exp 4, 6, 7): returns top-10 predictions PER HOP. Hop 0: top-10 relation candidates for the first step Hop 1: top-10 relation candidates for the second step etc.

```
def predict_exp9(rl_agent, tokenizer, question, device, max_hops=4, k=10):
    action_logits, _, rel_logits, _ = rl_agent(...)
    actions = torch.argmax(action_logits[0], dim=-1).tolist() # Best action per
hop
    for h in range(max_hops):
        a = actions[h]
        w = {0: 1, 1: 5, 2: 50}.get(a, 0) # Beam width from action
        if w > 0:
            _, topw = torch.topk(probs, k=w)
            results.append({'top_ids': topw.tolist()})
        else:
            break # STOP action
    return results
```

Exp 9 uses the RL AGENT'S CHOSEN ACTIONS to set the beam width dynamically. This is the actual inference behavior: some hops use 1 candidate, some use 5, some 50.

MEMORY OPTIMIZATION:

```
del model; torch.cuda.empty_cache() # After each model evaluation
```

Each model is deleted and GPU memory freed before loading the next. Without this: loading all 8 models would exceed VRAM.

RESULTS WRITING: Generates results.md with Markdown tables.

5.2 FILE: eval/universal_eval.py — Universal Dataset Evaluator

PURPOSE: Evaluate the Exp 10 Universal Planner after training is complete.

```

from train.exp10_universal import UniversalPlanner, UniversalDataset,
collate_universal, DATASET_IDS
model = UniversalPlanner(num_dom, num_rel).to(device)
model.load_state_dict(torch.load(args.ckpt))

for enc, doms, paths, nums, ds_ids in loader:
    out = model(enc['input_ids'], enc['attention_mask'], ds_ids)
    pred_rels = out['rel_logits'].argmax(dim=-1) # [B, max_hops]
    for b in range(B):
        target_path = paths[b, :nums[b]].tolist()
        pred_path = pred_rels[b, :nums[b]].tolist()
        if target_path == pred_path: # STRICT match
            correct += 1

```

STRICT PATH MATCHING: BOTH the relation IDs AND the order must match exactly. Gold: [312, 456] Pred: [312, 456] → CORRECT Pred: [312, 457] → WRONG (one relation off) Pred: [456, 312] → WRONG (correct relations, wrong order)

This is the same evaluation metric used in the CWQ experiments.

THREE SEPARATE EVALUATIONS:

```

results['cwq'] = evaluate_on_dataset(model, cwq_loader, ...)
results['webqsp'] = evaluate_on_dataset(model, webq_loader, ...)
results['metaqa'] = evaluate_on_dataset(model, meta_loader, ...)

```

Each dataset has its own test split loaded from `data/processed_universal/`. A SINGLE MODEL checkpoint is evaluated across all three simultaneously. This proves "universality" — the model didn't need separate specialized versions.

5.3 FILE: utils/verify.py — Checkpoint Verifier

Simple utility. Checks if checkpoint files exist on disk.

```

exps = {
    "Exp 0 (Flat Baseline)": "exp0_relation_flat_best.pt",
    "Exp 1 (Domain Baseline)": "exp1_domain_best.pt",
    "Exp 2 (CPD)": "exp2_cpd_best.pt",
    "Exp 3 (PCT)": "exp3_pct_best.pt",
    "Exp 4 (CHCP)": "exp4_chcp_best.pt"
}
for name, ckpt in exps.items():
    status = "LOADED" if os.path.exists(path) else "MISSING"
    print(f"{name} | Status: {status}")

```

Run this before evaluation to quickly confirm that all experiments have been trained. If a checkpoint is MISSING, the evaluation script will crash with a file-not-found error.

PART 6: RESULTS & PRODUCTION

6.1 ACTUAL ACCURACY NUMBERS (from results.md)

Path-Matching Evaluation (Exp 0-8, Planning Accuracy):

Model	Dev Hits@1	Test Hits@1	Dev Hits@3	Test Hits@3
Exp 0 (Flat Baseline)	32.53%	31.83%	42.97%	43.15%
Exp 3 (PCT Multi-task)	30.85%	26.85%	40.05%	38.18%
Exp 4 (CHCP Transformer)	56.20%	55.50%	73.50%	72.35%
Exp 4-RL (CHCP + RL)	23.50%	24.05%	52.74%	55.93%
Exp 6 (Unified)	53.60%	51.70%	71.30%	69.55%
Exp 7 (RoBERTa-L)	56.40%	57.28%	82.30%	81.70%
Exp 8 (CPD RoBERTa)	58.20%	56.76%	85.99%	85.24%

Execution-Based Evaluation (Exp 9, Paper-Comparable):

Model	Test Hits@1	Method
NSM (2021)	48.6%	Freebase Execution
SR+NSM (2022)	50.5%	Freebase Execution
TIARA (2022)	53.4%	Freebase Execution
ChatKBQA (2024)	55.5%	Freebase Execution
DRKG (2025)	66.9%	Freebase Execution
Exp 9 RLMC (Ours)	76.66%	Subgraph Execution

NOTE ON COMPARISON: "Subgraph Execution" = we traverse the CWQ-derived subgraph (a subset of Freebase). "Freebase Execution" = access to the full Freebase KB. Our SOTA result assumes the model accesses only the SPARQL-derived triples for each question.

Hop-Level Breakdown (Test Set):

Model	1-hop	2-hop	3-hop	4-hop
Exp 0	49.24%	28.57%	14.20%	5.58%
Exp 4	42.95%	61.81%	62.47%	54.82%

Model	1-hop	2-hop	3-hop	4-hop
Exp 8	49.33%	60.39%	64.71%	47.72%

OBSERVATION: Exp 0 was BETTER at 1-hop than Exp 4! WHY? For 1-hop questions, the simple flat classifier is "unencumbered" by the extra complexity of multi-hop planning. The Transformer adds OVERHEAD for simple cases. But for 2, 3, and 4-hop questions, Exp 4 is dramatically better.

6.2 PRODUCTION ENTITY LINKING STRATEGY

In the dataset: `item["topic_entity"]` gives us the Freebase ID directly. In production (real application): we must FIND this ID automatically.

PIPELINE:

Step 1: NAMED ENTITY RECOGNITION (NER) Input: "Who are the children of the president of the USA?" NER detects candidate entity mentions: ["the president", "USA"] NER tool: spaCy, or a BERT-based NER model fine-tuned on entity spans.

Step 2: ENTITY CANDIDATE RETRIEVAL (Dense Retrieval) For "USA": search a FAISS index of all Freebase entity embeddings. FAISS (Facebook AI Similarity Search): stores 100M+ entity vectors, finds top-50 nearest. Returns: [(m.09c7w0, "United States", 0.97), (m.0f8l9c, "USA (disambiguation)", 0.81), ...]

Step 3: ENTITY DISAMBIGUATION (Reranking) A cross-encoder or bi-encoder model sees the FULL QUESTION + each candidate. Scores each candidate: "In the context of 'president of the USA', which entity is right?" Selects: m.09c7w0 (the country, not a band or sports team).

Step 4: UNCERTAINTY HANDLING If top-1 confidence < threshold (e.g., 0.6): use TOP-3 candidates. Our set-based traversal starts from ALL 3 entities simultaneously. The KG execution naturally "kills off" wrong starts (they lead to dead ends). Only the entity that leads to a valid answer path "survives."

Tools: ELQ (Entity Linking for Questions): Fine-tuned on KGQA entity linking. BLINK (Facebook): Bi-encoder based, fast dense retrieval. GENRE (mBart-based): Generates entity names auto-regressively.

6.3 MATHEMATICAL GLOSSARY

Logit

Raw, unnormalized score output from the final linear layer. Range: $(-\infty, +\infty)$. The MAGNITUDE indicates confidence. Example: relation 312 has $\text{logit}=18.7$, all others ≈ -5 to 5 .

Softmax

Converts K logits to K probabilities summing to 1.0: $P(i) = \exp(\text{logit}_i) / \sum_j (\exp(\text{logit}_j))$ The exponentiation (exp) amplifies differences: if $\text{logit}_i \gg \text{logit}_j$, then $P(i) \approx 1$.

Cross Entropy Loss

$\text{CE}(\text{logits}, y) = -\log(\text{softmax}(\text{logits})[y])$ If model is 99% confident in correct class: $\text{CE} \approx 0.01$ (very small) If model is 1% confident in correct class: $\text{CE} \approx 4.6$ (very large) Minimizing CE = maximizing confidence in correct class.

Binary Cross Entropy (BCE)

For binary targets (0 or 1): $BCE = -[y \cdot \log(\sigma(x)) + (1-y) \cdot \log(1-\sigma(x))]$ $\sigma(x) = \text{sigmoid}(x) = 1/(1+\exp(-x))$: maps any number to $[0,1]$. Used for the Stop Token (should I stop here? binary decision per hop).

InfoNCE Loss

$L = -\log(\exp(\text{sim_pos}/\tau) / (\exp(\text{sim_pos}/\tau) + \sum_{\text{neg}} \exp(\text{sim_neg}/\tau)))$ Where τ (temperature) controls sharpness: Low τ (0.07): very sharp — model must be VERY confident about the positive. High τ (1.0): gentle — any reasonably high positive score is rewarded.

Advantage (RL)

$A(\text{state}, \text{action}) = R(\text{state}, \text{action}) - V(\text{state})$ $V(\text{state})$ = the VALUE NETWORK's estimate of total future reward from this state. $R(\text{state}, \text{action})$ = the ACTUAL reward obtained. $A > 0$: "This was better than expected → increase probability." $A < 0$: "This was worse than expected → decrease probability."

PPO Clip

Constrains policy updates to stay within $[1-\epsilon, 1+\epsilon]$ of the previous policy. Prevents catastrophic forgetting from a single lucky (or unlucky) batch.

Gradient Accumulation

Run N micro-batches, accumulating gradients after each, update weights once. Divide each micro-batch loss by N to maintain correct scaling. $\text{Effective batch_size} = \text{micro_batch_size} * N$.

Layer Normalization

For each sample independently: normalize activations to mean=0, std=1. Then scale by learned parameters γ (gamma) and β (beta). Prevents covariate shift, stabilizes training.

Bidirectional LSTM

Two LSTMs running in parallel: one left-to-right, one right-to-left. Final representation: concatenation of last hidden states from both directions. Captures context from BOTH sides of each element in the sequence.

Status: Exp 10 Universal Training — ACTIVE Stage 1 (CWQ) Epoch 7, Val Loss = 0.76 (was 0.99 at epoch 5) Goal: 861 relations, 70 domains, 3 datasets unified